Post-Mortem Analysis — Skill-It

Course: CS 320 — Software Engineering, Spring 2026 Instructor: Jaime Davila Team: Group 4 Project: Skill-It — Skill-Based Student Marketplace Development Period: February 4, 2026 – April 29, 2026 (85 days) Total Commits: ~112

---

1. Project Summary

Skill-It is a web-based marketplace where UMass students can post skill-based gigs and connect with peers to exchange services. Core functionality includes authentication (restricted to @umass.edu emails), job/gig posting, a messaging system with threaded conversations, user profiles with skill tagging, and a reviews/ratings system.
Tech Stack:

- Frontend: Next.js 16, React 19, TypeScript 5, Tailwind CSS 4
- Backend: Next.js API routes (serverless), Supabase (PostgreSQL + Auth)
- Testing: Jest 29, React Testing Library
- Other: Three.js / React Three Fiber (3D visuals), React Spring (animations)

---

2. What We Built

Feature	Status
Email auth (UMass-only @umass.edu)	Complete
User profiles with skill tagging	Complete
Profile picture uploads	Complete
Job/gig posting with custom skills and categories	Complete
Messaging system with thread archival	Complete
Unread message notifications	Complete
Reviews and ratings	Complete
Light/dark mode	Complete
Launch/landing page with 3D visuals	Complete
Unit tests (7 test suites)	Partial

---

3. What Went Well

Team Coordination

We used a consistent feature-branch and pull-request workflow throughout the project. With 8 contributors landing 49 PRs, parallel development stayed largely conflict-free. Code review through PRs caught issues before they hit main.

Scope Management

We identified a clear MVP early (auth, jobs, messaging, profiles) and executed on it before adding polish (reviews, dark mode, 3D landing page). This ordering meant we always had a working product even as features were in progress.

Tech Stack Choices

Using Supabase for both the database and authentication reduced infrastructure overhead significantly. Next.js API routes kept the backend and frontend in one repository, simplifying deployment and local development setup. TypeScript caught a number of bugs at compile time that would have been runtime issues in plain JavaScript.

Feature Delivery

All major planned features shipped: auth, profiles, jobs, messaging, and reviews. The reviews system — the last major feature — merged on April 29, the day before the final deadline.

---

4. What Didn't Go Well

Testing Strategy Was an Afterthought

Unit tests were not added until mid-April (commits April 16–17), roughly 75% of the way through the project. The reviews feature explicitly noted "still need testing" in its initial commit (April 21) and was merged before that testing was complete. We ended up with 7 test suites covering the main entities but without consistent coverage across all API endpoints and edge cases.
Root cause: We treated testing as a phase to do after features rather than alongside them. No test-writing was planned into sprint work.
What to do differently: Define a "done" standard that includes tests. Write tests for each API route and service function as the route is written, not after.

UI Was Unstable and Required a Major Reset

A single commit on April 15 describes itself as "Major changes in UI. We can re-add detail later." This signals that the UI had accumulated enough complexity or inconsistency that the fastest path forward was to strip it back and rebuild. That's a significant context switch mid-sprint.

Root cause: UI decisions were made incrementally without an agreed-upon design reference. Individual contributors had different interpretations of the visual direction, which created drift.
What to do differently: Agree on a component/layout library or a low-fidelity wireframe early and use it as the source of truth for UI decisions. This doesn't require a dedicated designer — even a rough Figma sketch prevents divergence.

Pull Request Descriptions Were Minimal

Most PRs (and many commits) had single-line descriptions: "Edited job page," "fixed multiple review viewing," "major ui changes." For a CS 320 course project with graded artifacts, richer descriptions would have made code review more effective and the git history more useful as documentation.
Root cause: No team norm was set for what a PR description should include.
What to do differently: Establish a simple PR template at the start of the project: what changed, why, and how to test it. Three sentences per PR would have been enough.

Merge Conflicts and Branch Overhead

Several commits exist solely to merge main back into a feature branch mid-work. This is a normal part of parallel development, but the frequency suggests some branches ran longer than necessary before integration.
Root cause: Some feature branches were large in scope and took too long to land. The longer a branch lives, the more drift accumulates against main.
What to do differently: Break large features into smaller, mergeable slices. A branch that represents two or three days of work is far easier to integrate than one representing two weeks.

---

5. Timeline Reflection

Period	Work Done	Notes
Feb 4 – Mar 2	Planning, user stories, ethics assignment, DB diagram	Good front-loaded planning
Mar 2 – Apr 6	Schema, API foundations, auth	Solid core; some gitignore issues
Apr 6 – Apr 15	Profiles, messaging, settings, notifications	Productive sprint; UI drift begins
Apr 15 – Apr 22	UI overhaul, job enhancements, email restrictions, tests	Significant rework cost
Apr 22 – Apr 29	Reviews, dark mode, archive feature, launch page	Strong final push

The final week delivered five distinct features, which suggests some features that should have landed earlier got pushed to the end. The reviews system in particular is complex enough that it warranted more runway.

---

6. Technical Debt Left Behind

1. Incomplete test coverage — the reviews service and several API routes lack tests. Any future change to review logic carries regression risk.
2. "Re-add detail later" UI — the April 15 UI simplification was a deferral, not a decision. Several UI refinements noted as future work were never revisited.
3. No end-to-end tests — unit tests cover individual functions, but there is no automated test that exercises the full user flow (sign up → post a job → message → review).
4. Custom categories not fully integrated — the new_categories branch added the ability to create categories, but it's unclear whether this is surfaced consistently in job browsing and filtering.

---

7. If We Did It Again

1. Write tests as features are built, not after. The cost of retrofitting tests is always higher than writing them alongside the feature.
2. Set a UI direction in week one. Even a rough wireframe prevents the expensive mid-project reset we experienced.
3. Timebox branches. Any branch that lives more than four or five days should be broken up or merged as a work-in-progress.
4. Use PR templates. Three sentences per PR — what, why, how to test — is a minimal investment with a large return in review quality and historical clarity.
5. Plan the reviews system earlier. It was one of the most complex features and was the last to land. Core features should be sequenced before stretch features.

---

8. Closing Notes

The project successfully delivered a working, multi-user web application with authentication, data persistence, real-time-adjacent messaging, and a reviews system — in 85 days with a team of 8, all while managing a full course load. The product works. The patterns and processes around building it have clear room to improve, which is the point of a post-mortem.

The main lesson is not about any single bug or missed feature. It's that the overhead of coordination — keeping branches short, writing tests alongside code, agreeing on visual direction early — pays for itself quickly on a team of this size. These habits don't slow down development; they prevent the catch-up work that slows development.